

ASSIGNMENT – 2.1

NAME : G.RISHITHA

HT.NO : 2303A52149

BATCH : 35

Lab 2: Exploring Additional AI Coding Tools beyond Copilot – Gemini (Colab) and Cursor AI.

Task 1: Statistical Summary for Survey Data

Scenario: You are a data analyst intern working with survey responses stored as numerical lists.

Task: Use Google Gemini in Colab to generate a Python function that reads a list of numbers and calculates the mean, minimum, and maximum values.

Expected Output:

Correct Python function

Output shown in Colab

Screenshot of Gemini prompt and result

AI_Assistant_Coding.ipynb - Colab

colab.research.google.com/drive/198G0BRUSAkmytLA5i8BnXC9hBA8R3mN4

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

Connect

Gemini

```
def calculate_stats(numbers):  
    """  
    Calculates the mean, minimum, and maximum of a list of numbers.  
    Args:  
        numbers (list): A list of numerical values.  
    Returns:  
        tuple: A tuple containing (mean, minimum, maximum).  
        Returns (None, None, None) if the list is empty.  
    """  
    if not numbers:  
        return None, None, None  
    mean_value = sum(numbers) / len(numbers)  
    min_value = min(numbers)  
    max_value = max(numbers)  
    return mean_value, min_value, max_value  
  
# Example usage:  
my_list = [10, 20, 30, 40, 50]
```

generate a Python function that reads a list of numbers and calculates the mean, minimum, and maximum values

Sure, here's a Python function that takes a list of numbers and returns its mean, minimum, and maximum values. I've also included an example of how to use it.

Accept & Run Accept Cancel

What can I help you build?

Gemini 2.5 Flash

Gemini can make mistakes so double-check it and use code with caution. [Learn more](#)

Variables Terminal

80° Search

ENG IN 01:57 PM 12-01-2026

AI_Assistant_Coding.ipynb - Colab

colab.research.google.com/drive/198G0BRUSAkmytLA5i8BnXC9hBA8R3mN4#scrollTo=58fc6def

Commands + Code + Text Run all

RAM Disk

[3]

```
def calculate_stats(numbers):  
    """  
    Calculates the mean, minimum, and maximum of a list of numbers.  
    Args:  
        numbers (list): A list of numerical values.  
    Returns:  
        tuple: A tuple containing (mean, minimum, maximum).  
        Returns (None, None, None) if the list is empty.  
    """  
    if not numbers:  
        return None, None, None  
    mean_value = sum(numbers) / len(numbers)  
    min_value = min(numbers)  
    max_value = max(numbers)  
    return mean_value, min_value, max_value  
  
# Example usage:  
my_list = list(map(int, input().split()))  
mean, minimum, maximum = calculate_stats(my_list)  
  
print(f"List: {my_list}")  
print(f"Mean: {mean}")  
print(f"Minimum: {minimum}")
```

Variables Terminal

1:58 PM Python 3

80° Search

ENG IN 01:59 PM 12-01-2026

The screenshot shows a Google Colab notebook with the following code and output:

```
min_value = min(numbers)
max_value = max(numbers)

return mean_value, min_value, max_value

# Example usage:
my_list = list(map(int, input().split()))
mean, minimum, maximum = calculate_stats(my_list)

print(f"List: {my_list}")
print(f"Mean: {mean}")
print(f"Minimum: {minimum}")
print(f"Maximum: {maximum}")

empty_list = []
mean_empty, min_empty, max_empty = calculate_stats(empty_list)
print(f"Empty List: {empty_list}")
print(f"Mean: {mean_empty}, Min: {min_empty}, Max: {max_empty}")
```

Output:

```
110 20 30 40 50
List: [10, 20, 30, 40, 50]
Mean: 30.0
Minimum: 10
Maximum: 50

Empty List: []
Mean: None, Min: None, Max: None
```

Task 2: Armstrong Number – AI Comparison

Scenario: You are evaluating AI tools for numeric validation logic.

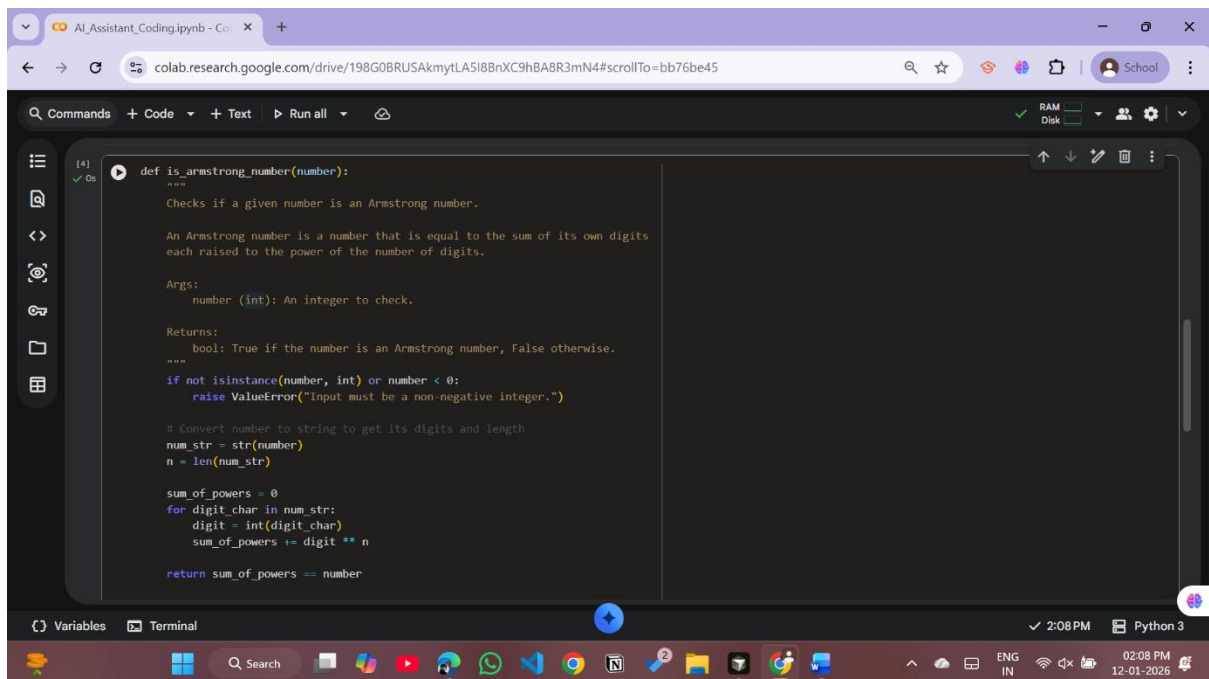
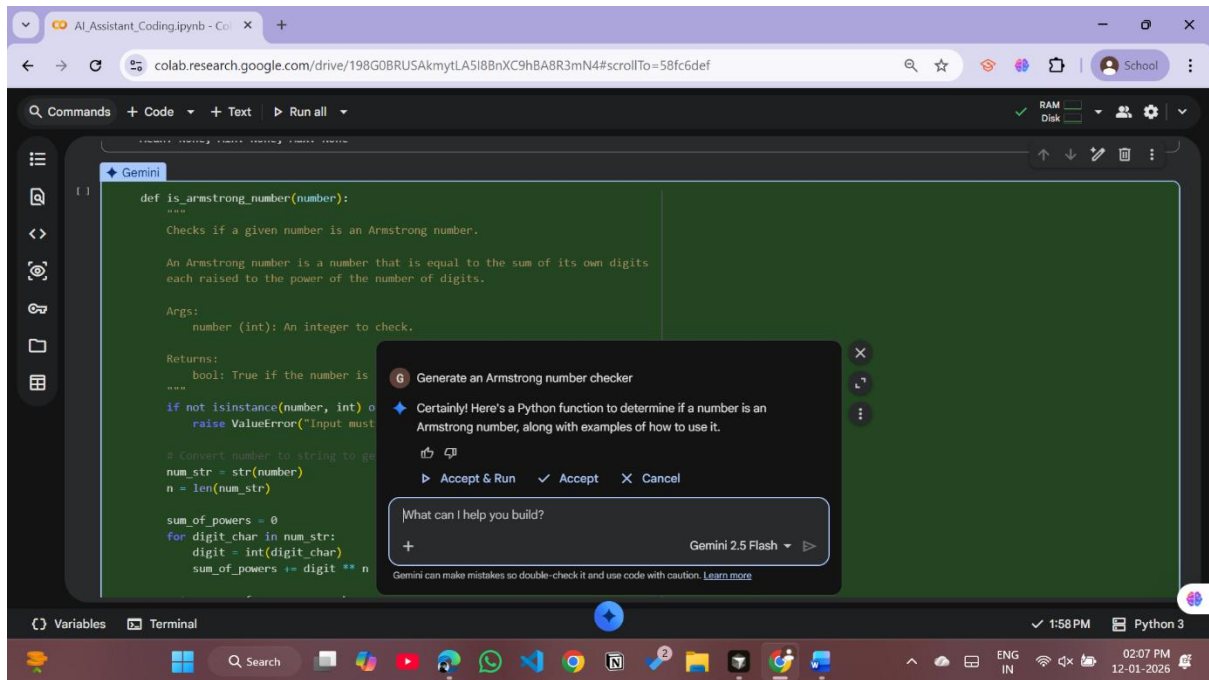
Task: Generate an Armstrong number checker using Gemini and GitHub Copilot. Compare their outputs, logic style, and clarity.

Expected Output:

Side-by-side comparison table

Screenshots of prompts and generated code.

Gemini (GoogleColab):



The screenshot shows a Google Colab notebook with a Python function `is_armstrong_number` and its execution output. The function takes a number as input and returns a boolean indicating if it is an Armstrong number. The output shows the function being called with various numbers and the expected results.

```
[4] 0s
def is_armstrong_number(number):
    sum_of_powers = 0
    for digit_char in str(number):
        digit = int(digit_char)
        sum_of_powers += digit ** len(str(number))

    return sum_of_powers == number

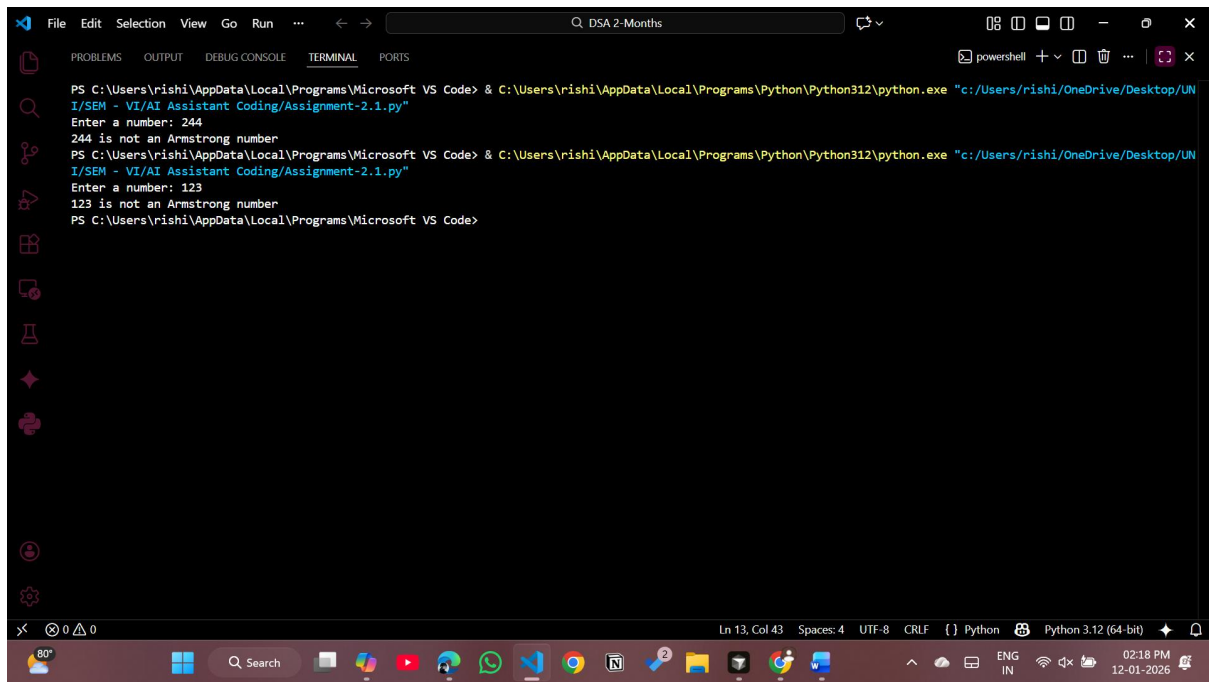
# Example usage:
print(f"Is 153 an Armstrong number? {is_armstrong_number(153)}") # Expected: True
print(f"Is 9 an Armstrong number? {is_armstrong_number(9)}") # Expected: True
print(f"Is 370 an Armstrong number? {is_armstrong_number(370)}") # Expected: True
print(f"Is 371 an Armstrong number? {is_armstrong_number(371)}") # Expected: True
print(f"Is 1634 an Armstrong number? {is_armstrong_number(1634)}") # Expected: True
print(f"Is 123 an Armstrong number? {is_armstrong_number(123)}") # Expected: False
print(f"Is 0 an Armstrong number? {is_armstrong_number(0)}") # Expected: True

... Is 153 an Armstrong number? True
Is 9 an Armstrong number? True
Is 370 an Armstrong number? True
Is 371 an Armstrong number? True
Is 1634 an Armstrong number? True
Is 123 an Armstrong number? False
Is 0 an Armstrong number? True
```

GithubCopilot (VSCode):

The screenshot shows a VS Code editor with a Python script for checking Armstrong numbers. The script prompts the user to enter a number and then checks if it is an Armstrong number. The output shows the number being checked and the result.

```
File Edit Selection View Go Run ...
C:\Users\> Users > OneDrive > Desktop > UNI > SEM - VI > AI Assistant Coding > Assignment-2.1.py > ...
1 #Armstrong Number Checker
2 num = int(input("Enter a number: "))
3 order = len(str(num))
4 sum = 0
5 temp = num
6 while temp > 0:
7     digit = temp % 10
8     sum += digit ** order
9     temp //= 10
10 if num == sum:
11     print(num, "is an Armstrong number")
12 else:
13     print(num, "is not an Armstrong number")
```



```
PS C:\Users\rishi\AppData\Local\Programs\Microsoft VS Code> & C:\Users\rishi\AppData\Local\Programs\Python\Python312\python.exe "c:/Users/rishi/OneDrive/Desktop/UNI/SEM - VI/AI Assistant Coding/Assignment-2.1.py"
Enter a number: 244
244 is not an Armstrong number
PS C:\Users\rishi\AppData\Local\Programs\Microsoft VS Code> & C:\Users\rishi\AppData\Local\Programs\Python\Python312\python.exe "c:/Users/rishi/OneDrive/Desktop/UNI/SEM - VI/AI Assistant Coding/Assignment-2.1.py"
Enter a number: 123
123 is not an Armstrong number
PS C:\Users\rishi\AppData\Local\Programs\Microsoft VS Code>
```

Improvements after comparing Gemini and GitHub Copilot outputs:

- The AI-generated code improved **logical clarity**, making the Armstrong calculation easier to understand.
- Variable naming became more meaningful, which improved **code readability**.
- The refactored version reduced unnecessary steps, making the program **more efficient**.
- Use of loops and arithmetic operations was optimized for better **performance**.
- Overall structure became more **beginner-friendly**, helping in learning numeric validation logic.

Task 3: Leap Year Validation Using Cursor AI

Scenario: You are validating a calendar module for a backend system.

Task: Use Cursor AI to generate a Python program that checks whether a given year is a leap year. Use at least two different prompts and observe changes in code.

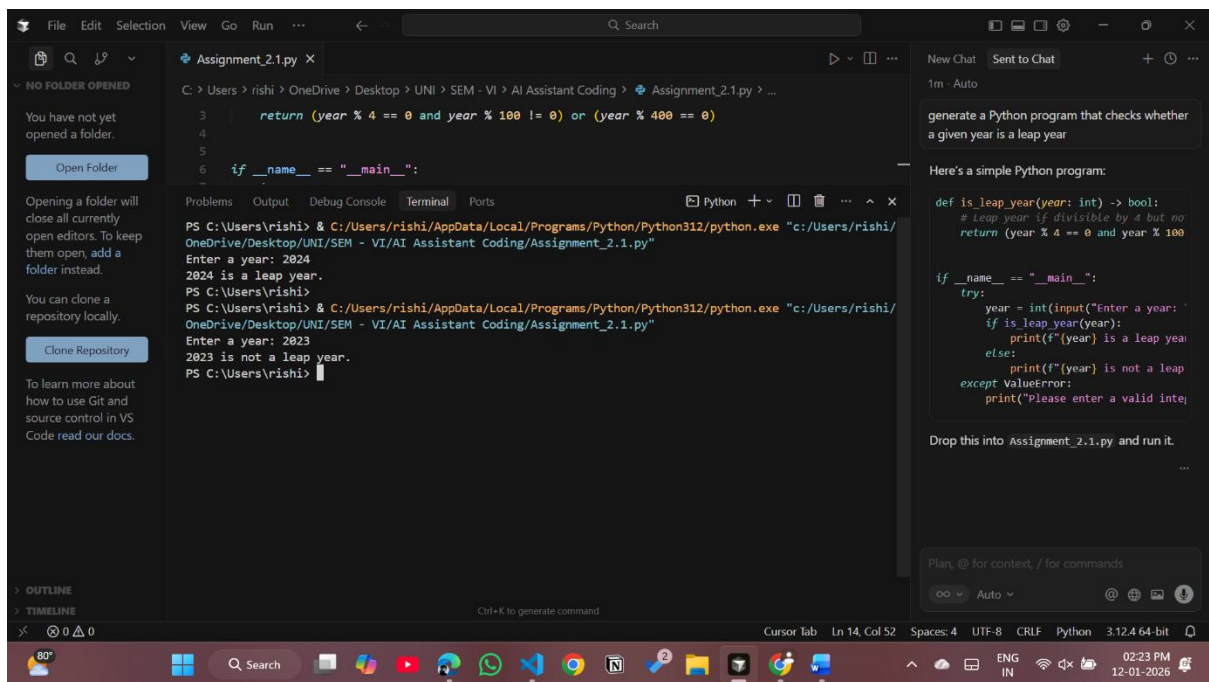
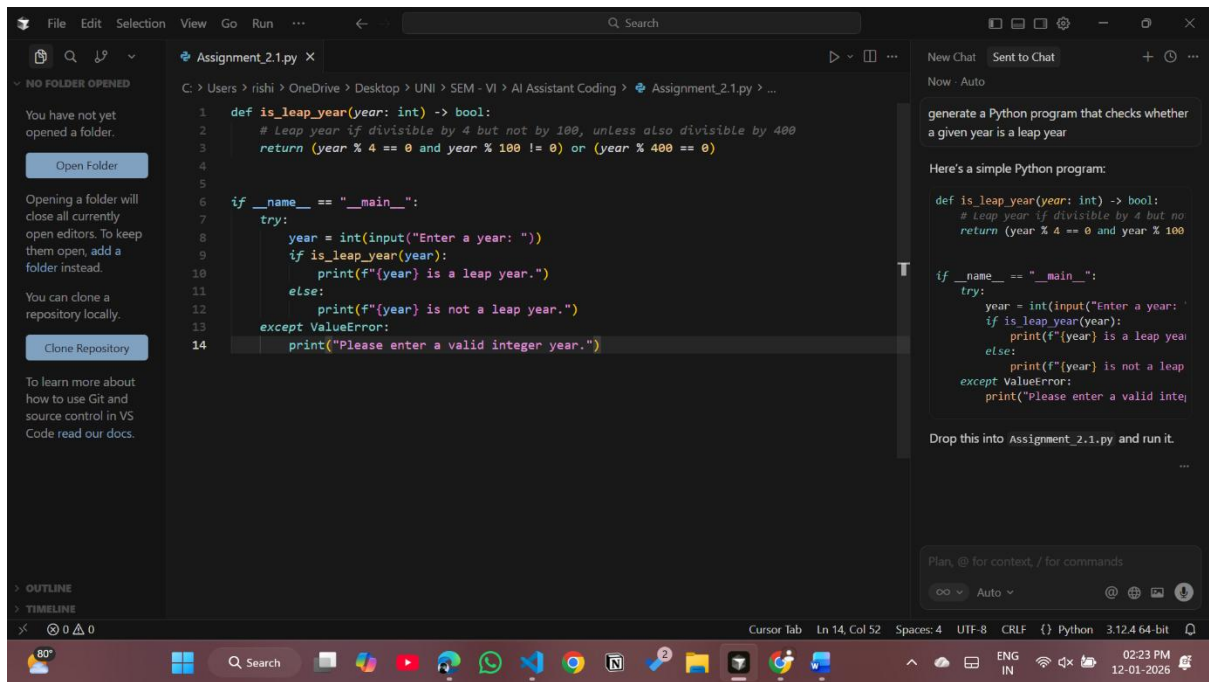
Expected Output:

Two versions of code

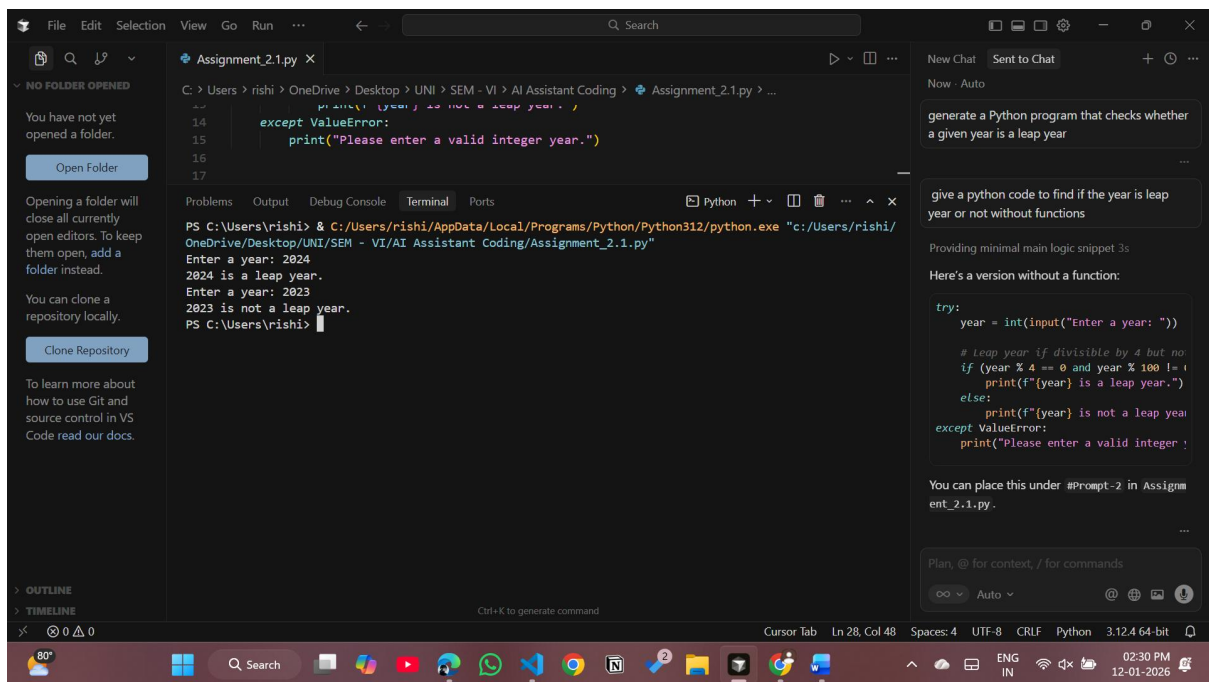
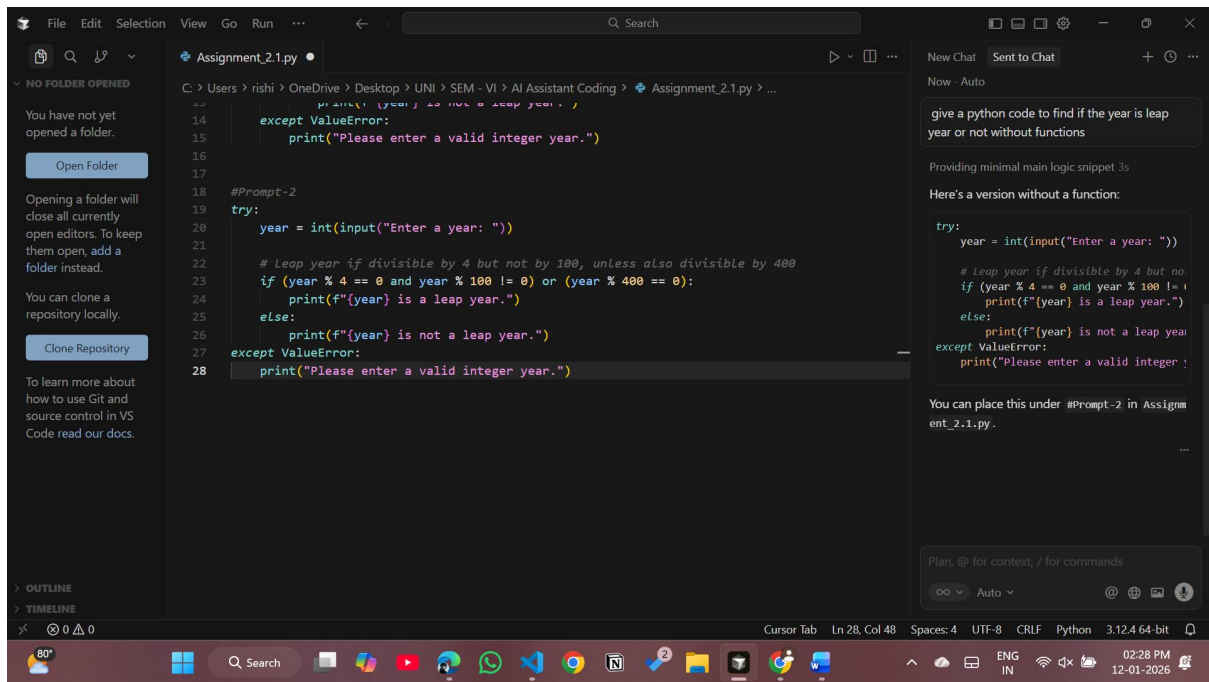
Sample inputs/outputs

Brief comparison

Prompt-1: generate a Python program that checks whether a given year is a leap year.



Prompt-2: give a python code to find if the year is leap year or not without functions.



Improvements observed across different prompts:

- Changing the prompt resulted in **clearer logical flow** in the second version of the code.
- Step-by-step conditional checks improved **readability and understanding**.
- The refined code avoided ambiguity in leap year rules, ensuring **logical correctness**.
- Prompt variation demonstrated how AI can adapt logic based on **developer intent**.
- The final code became easier to test and debug due to **explicit conditions**.

Task 4: Student Logic + AI Refactoring (Odd/Even Sum)

Scenario: Company policy requires developers to write logic before using AI.

Task: Write a Python program that calculates the sum of odd and even numbers in a tuple, then refactor it using any AI tool.

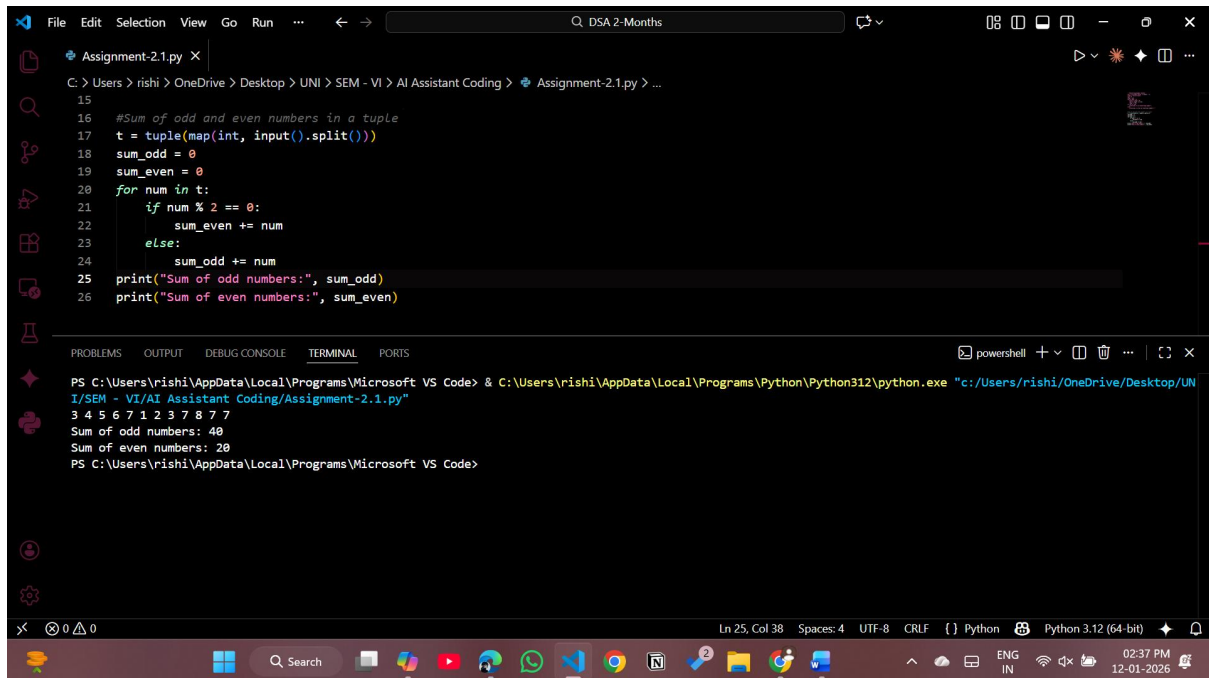
Expected Output:

Original code

Refactored code

Explanation of improvements

Original Code:



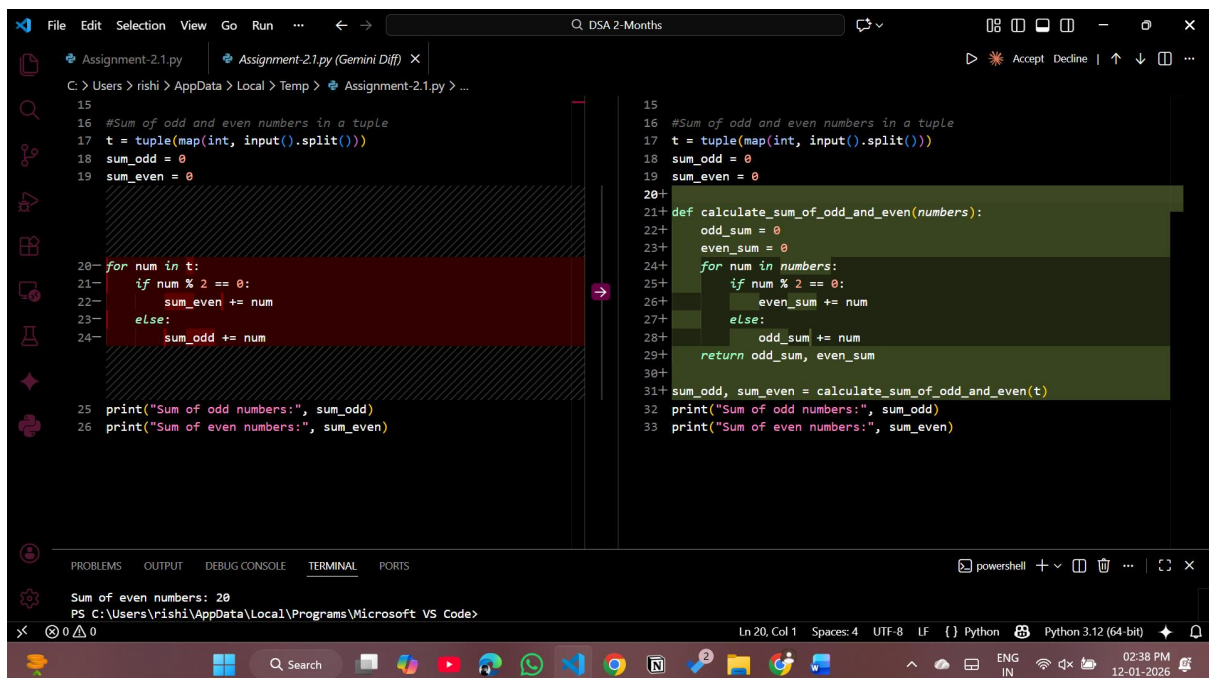
The screenshot shows a VS Code editor with a file named 'Assignment-2.1.py'. The code is a Python script that takes a tuple of numbers as input and calculates the sum of odd and even numbers. The code is as follows:

```
15
16 #Sum of odd and even numbers in a tuple
17 t = tuple(map(int, input().split()))
18 sum_odd = 0
19 sum_even = 0
20 for num in t:
21     if num % 2 == 0:
22         sum_even += num
23     else:
24         sum_odd += num
25 print("Sum of odd numbers:", sum_odd)
26 print("Sum of even numbers:", sum_even)
```

The terminal output shows the execution of the script with the input '3 4 5 6 7 1 2 3 7 8 7 7'.

```
PS C:\Users\rishi\AppData\Local\Programs\Microsoft VS Code> & C:\Users\rishi\AppData\Local\Programs\Python\Python312\python.exe "c:/Users/rishi/OneDrive/Desktop/UNI/SEM - VI/AI Assistant Coding/Assignment-2.1.py"
3 4 5 6 7 1 2 3 7 8 7 7
Sum of odd numbers: 40
Sum of even numbers: 20
PS C:\Users\rishi\AppData\Local\Programs\Microsoft VS Code>
```

Refactored Code:

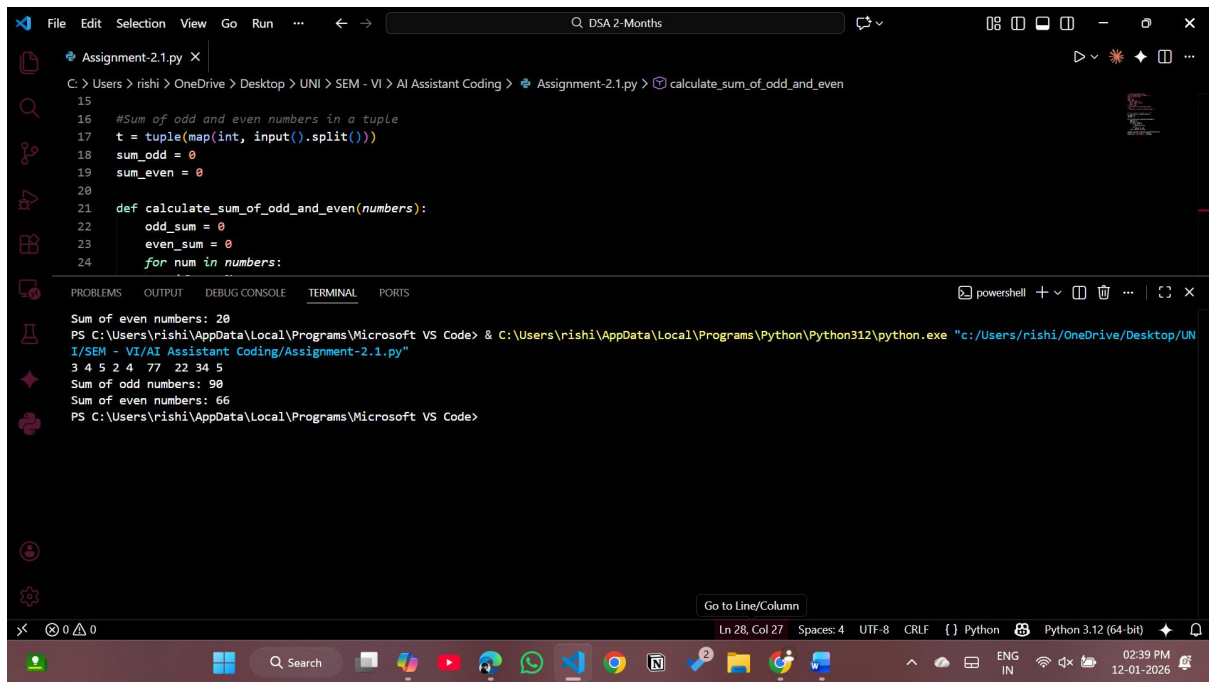


The screenshot shows a VS Code editor with a file named 'Assignment-2.1.py'. The code is a Python script that takes a tuple of numbers as input and calculates the sum of odd and even numbers. The code is as follows:

```
15
16 #Sum of odd and even numbers in a tuple
17 t = tuple(map(int, input().split()))
18 sum_odd = 0
19 sum_even = 0
20
21 def calculate_sum_of_odd_and_even(numbers):
22     odd_sum = 0
23     even_sum = 0
24     for num in numbers:
25         if num % 2 == 0:
26             even_sum += num
27         else:
28             odd_sum += num
29     return odd_sum, even_sum
30
31 sum_odd, sum_even = calculate_sum_of_odd_and_even(t)
32 print("Sum of odd numbers:", sum_odd)
33 print("Sum of even numbers:", sum_even)
```

The terminal output shows the execution of the script with the input '3 4 5 6 7 1 2 3 7 8 7 7'.

```
Sum of even numbers: 20
PS C:\Users\rishi\AppData\Local\Programs\Microsoft VS Code>
```



The screenshot shows a Visual Studio Code editor window with a file named 'Assignment-2.1.py'. The code in the editor is as follows:

```
15
16 #Sum of odd and even numbers in a tuple
17 t = tuple(map(int, input().split()))
18 sum_odd = 0
19 sum_even = 0
20
21 def calculate_sum_of_odd_and_even(numbers):
22     odd_sum = 0
23     even_sum = 0
24     for num in numbers:
```

The terminal at the bottom shows the execution of the script. It prompts for input, which is '3 4 5 2 4 77 22 34 5'. The output shows the sum of even numbers as 20 and the sum of odd numbers as 90. The terminal also shows the command used to run the script: `python.exe "c:/Users/rishi/OneDrive/Desktop/UNI/SEM - VI/AI Assistant Coding/Assignment-2.1.py"`.

Improvements after AI-based refactoring:

- Logic was modularized using a function, improving **code reusability**.
- The refactored code follows **better programming practices**.
- Maintenance became easier since changes are limited to a single function.
- Improved structure enhanced **readability and scalability**.
- Demonstrates how AI helps refine existing logic without changing functionality.

